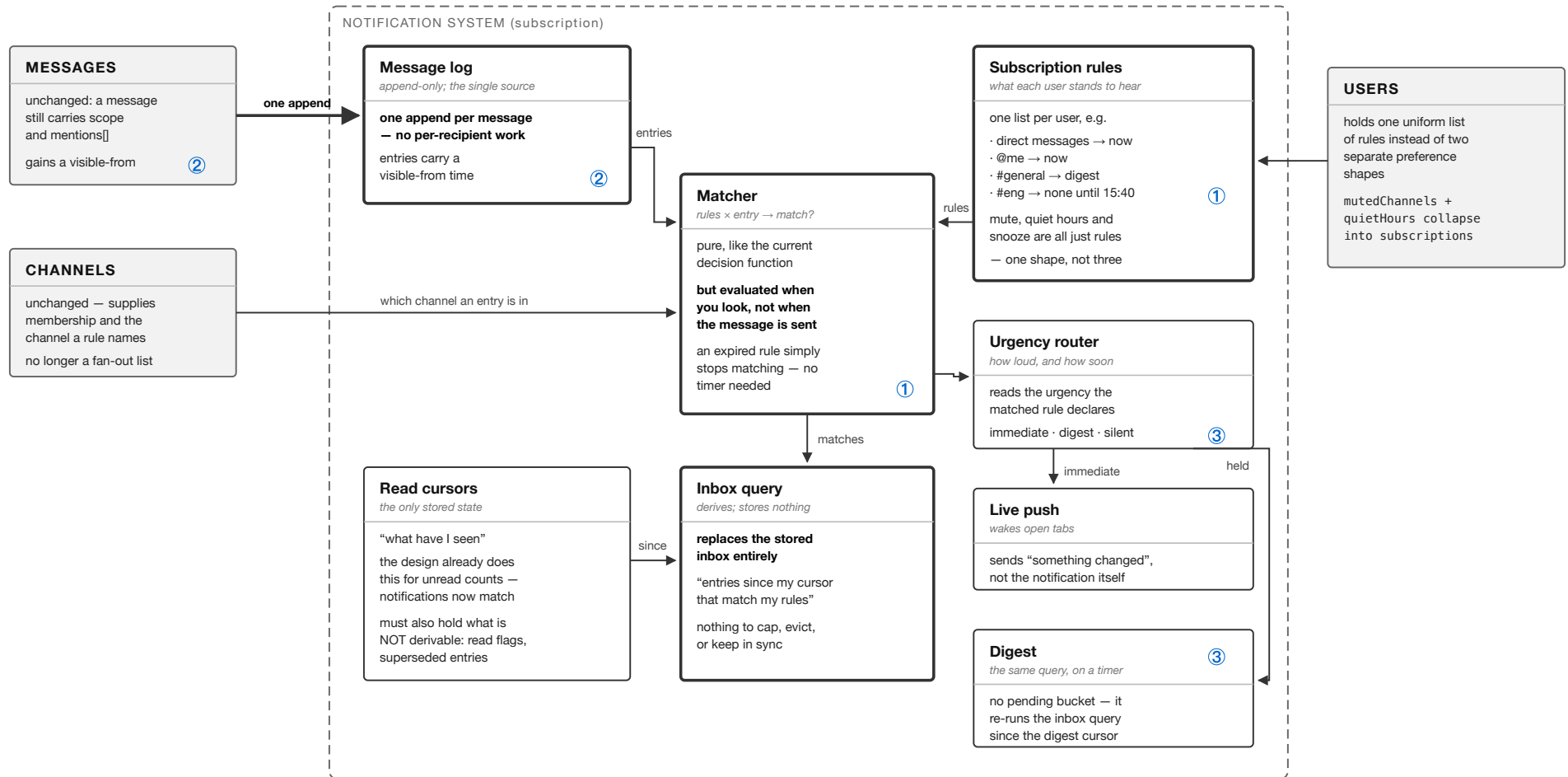


A1 — Alternative design: notifications by subscription, not by push

Same boundary, same three connection points. The one decision changed: **the decision moves from write time to read time**. A message is appended once; a user's notifications are **derived** from their standing rules instead of written into a per-user inbox. Components are labelled by responsibility (not implemented).



1 · What I changed: when the decision is made

The generated design decides once, at send time, and stores the answer per recipient. This one stores only the rules and the log, and works the answer out when you look. Preferences are consulted at read time, so the version of your settings that counts is the one in force *now* — not the one that happened to be in force when someone else pressed send.

2 · Which quality it improves: changeability

Each of the three requirements got smaller. Snooze: the largest piece of work was a client timer to stop a lapsed mute looking live — with nothing cached, nothing goes stale, and that work disappears. Digest: no held-item flag, no pending bucket, no flush — a digest is the same query run from a different cursor.

3 · What it costs: the past stops holding still

Deciding at read time means changing a rule changes history. Mute a channel today and yesterday's inbox quietly rearranges itself, because it is recomputed rather than recalled. A stored notification records what was true when it was sent; a derived one is not a record at all.

Legend

→ data flow, labelled
bold changed from the as-built
grey box outside the boundary
① ② ③ where change 1 (snooze), 2 (scheduled) and 3 (digest) would land — draft, check it